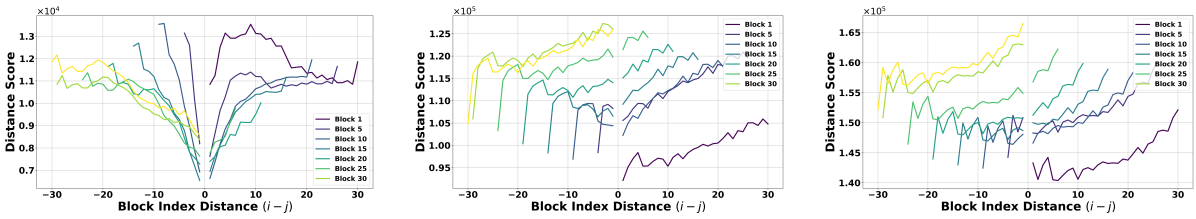

(a) Eq. (2) *with* high-rank pruning (b) Eq. (2) *without* high-rank pruning (c) Frobenius norm of $\mathbf{W}_i - \mathbf{W}_j$

Figure 2: Comparison of block distance score versus block index distance $(i - j)$ for different metrics. (a) Using the proposed metric in Eq. (2) with high-rank pruning, showing that closer blocks score lower (better), matching our intuition that weights close in the model have similar function. (b) Ablation of high-rank pruning, where there is no clear trend except that blocks closer to 0 are lower and those closer to 31 are higher. (c) Simple Frobenius norm, showing a similar lack of clear trend as in (b). *We found that using the score in (a) as the weight-sharing selection metric results in a much higher performing model compared to using the scores in (b) and (c).*

as the Frobenius norm in the weight space often results in suboptimal selections. Specifically, we find that *all blocks 'choose' a single block*, whereas intuition suggests a diverse selection of base blocks would work better[1].

Instead, we employ a selection metric based on *low-rank SVD reconstructions* to achieve a more effective and intuitive solution. Utilizing low-rank approximations, namely $\hat{\mathbf{W}}_i$ and $\hat{\mathbf{W}}_j$, instead of directly using $\mathbf{W}_i$ and $\mathbf{W}_j$ helps avoid the pitfall where all pruned blocks are replaced by a single block. We believe high-rank elimination is beneficial because low-rank approximations capture the most significant components of the weights, thereby simplifying the process of identifying suitable replacements by eliminating high-rank 'noise'. Our method reveals that blocks nearest to the pruned blocks in the model tend to have the lowest scores, indicating higher similarity.

The distance metric $d(\mathbf{W}_i, \mathbf{W}_j)$ for selecting the replacement block is defined as:

$$d(\mathbf{W}_i, \mathbf{W}_j) = \left\| \hat{\mathbf{W}}_i - \left(\hat{\mathbf{W}}_j + \Delta_{i-j} \right) \right\|_F \quad (2)$$

where:

- $\hat{\mathbf{W}}_i$ and $\hat{\mathbf{W}}_j$ are the low-rank SVD reconstructions of $\mathbf{W}_i$ and $\mathbf{W}_j$, respectively, using the first r ranks.
- $\Delta_{i-j} \triangleq (\mathbf{U}_{i-j}\mathbf{\Sigma}_{i-j})[1 : r](\mathbf{V}_{i-j}[1 : r])^T$ is the rank-r approximation of the difference $\hat{\mathbf{W}}_i - \hat{\mathbf{W}}_j$. We used a rank of $r = 256$.
- $\| \cdot \|_F$ denotes the Frobenius norm.

[1]We empirically demonstrate this in the experiments section in the first row of Table 4

These low-rank approximations $\hat{\mathbf{W}}_i$ and $\hat{\mathbf{W}}_j$ are obtained via:

$$\hat{\mathbf{W}}_i = (\mathbf{U}_i\mathbf{\Sigma}_i)[1 : r](\mathbf{V}_i[1 : r])^T \quad (3)$$

$$\hat{\mathbf{W}}_j = (\mathbf{U}_j\mathbf{\Sigma}_j)[1 : r](\mathbf{V}_j[1 : r])^T \quad (4)$$

Finally, for each pruned block i, we select its base for weight sharing as the candidate block j with the minimum score in (2):

$$j = \underset{j' \neq i}{\operatorname{argmin}}\, d(\mathbf{W}i, \mathbf{W}j') \quad (5)$$

This approach is highly intuitive, as proximal blocks are naturally more alike. In Figure 2(a), the proposed metric with high-rank pruning, Eq. (2), shows that blocks closer in the model score lower (better), confirming our intuition that proximate blocks have similar functions. Figures 2(b) and 2(c), which respectively ablate high-rank pruning and use simple Frobenius norm, lack this clear trend, and furthermore we found that they result in significantly weaker models. An alternative version of this Figure is available in the Appendix, where the x-axis is candidate block index j instead of block index distance $(i - j)$.

3.3 Output Normalization

We apply layer normalization (Ba et al., 2016) to the output of each MHSA and MLP layer in the weight-sharing layers, specifically to the previously pruned blocks. This normalization is applied across the hidden state dimension and is initialized to a small value set by a hyperparameter, allowing the model to gradually learn and adjust the output magnitudes over time. The normalized output $\mathbf{h}_{norm}$ is defined as:

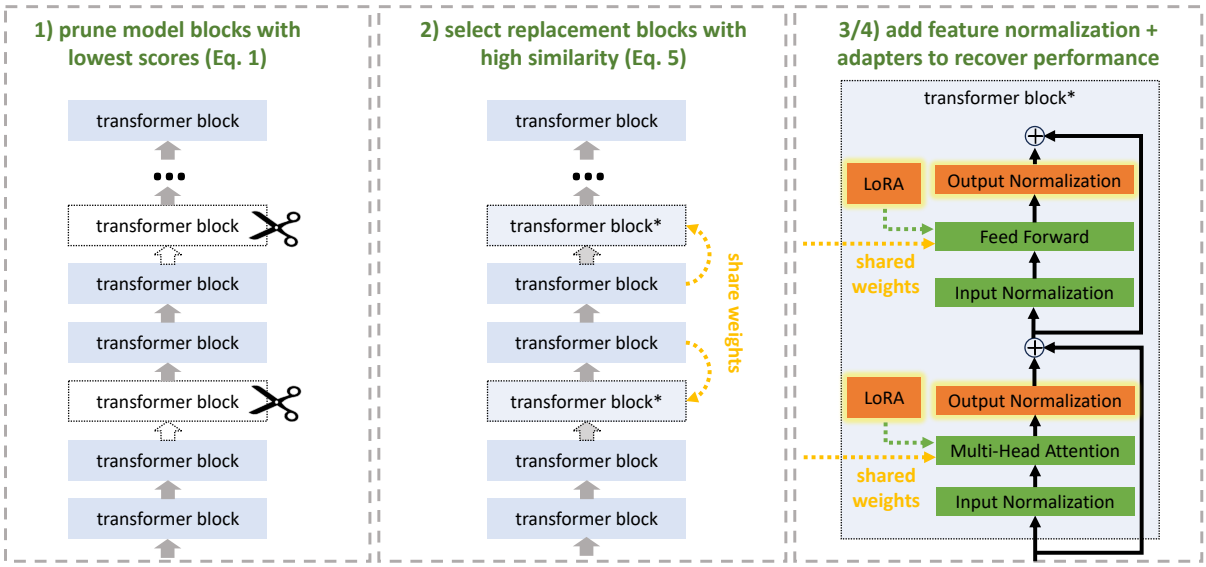

Figure 3: Overview of the FlexiGPT pruning process. **Left:** We prune model blocks with the lowest scores based on (1). **Center:** We select replacement blocks with high similarity using (5). **Right:** We add feature normalization and learn adapters to recover performance.

$$\mathbf{h}_{norm} = \frac{\mathbf{h} - \mu(\mathbf{h})}{\sigma(\mathbf{h})} \times \gamma \tag{6}$$

where:

- $\mathbf{h}$ is the output of the layer before normalization.
- $\mu(\mathbf{h})$ and $\sigma(\mathbf{h})$ are the mean and standard deviation of $\mathbf{h}$, respectively.
- γ is a learnable scaling weight of the same dimension as the model hidden state size.

This approach is akin to initializing the B matrix in LoRA (Hu et al., 2021) such that $\Delta W = BA$ is zero at the beginning of training. This similarity arises because both methods aim to minimize initial disruptions to the model and allow gradual learning. In LoRA, initializing $\Delta W = BA$ to zero helps avoid high initial loss, ensuring smoother training. Similarly, by initializing the hidden states of weight-shared blocks to small values, we avoid significant jumps in PPL at the start of training. As shown in Table 4, this approach is crucial for maintaining low PPL post-pruning and ensuring stable model performance during fine-tuning.

3.4 Adapters and Initialization

We employ LoRA to facilitate weight sharing for the pruned blocks, providing a parameter-efficient mechanism to adjust the weights of the replaced blocks. The LoRA adapters consist of two low-rank matrices, A and B, inserted into the linear transformations of the shared weights in the model, effectively increasing the expressive capacity of these blocks despite the weight-sharing constraint[2]. The weights of the adapters are initialized using the SVD between the pruned block and its replacement block, as described in the selection of weight-sharing bases. Specifically, we decompose the difference between the pruned block $\mathbf{W}_i$ and the replacement block $\mathbf{W}_j$ into low-rank matrices:

$$\mathbf{W}_i - \mathbf{W}_j = \mathbf{U}_{i-j}\mathbf{\Sigma}_{i-j}\mathbf{V}_{i-j}^T \tag{7}$$

The adapter matrices A and B are then initialized as:

$$A = (\mathbf{U}_{i-j}\mathbf{\Sigma}_{i-j})[1:r], B = (\mathbf{V}_{i-j}[1:r])^T \tag{8}$$

where:

- $(\mathbf{U}_{i-j}\mathbf{\Sigma}_{i-j})[1:r]$ is the product of the left singular vectors and the diagonal matrix of singular values, indexed to take the first r columns.
- $(\mathbf{V}_{i-j}[1:r])^T$ is the transposed matrix containing the first r columns of the right singular vectors.

[2]We also 'unlock' the shared weight during training.

Our method requires a small amount of post-pruning fine-tuning to fully recover performance, which is discussed in Section 4. However, we generally observe that the post-prune PPL is indicative of which method will finish with a lower PPL. In Table 4, we see the effect of output normalization and LoRA initialization on post-prune PPL. While the SVD initialization is of smaller yet significant importance to our method, the output normalization, initialized to a small value to minimize initial disruptions and allow gradual learning, is crucially important. This is evident from the drastic increase in post-prune PPL when output normalization is ablated. The combination of SVD initialization and carefully tuned output normalization ensures that our method maintains low perplexity and stable performance during the fine-tuning phase.

3.5 Model Extension

In addition to pruning, FlexiGPT can also be used to extend smaller models, such as a 22-layer TinyLLaMA (Zhang et al., 2024). In this second setting, we repeat blocks in a sequence determined by hyperparameter indexes that denote the start and end of the repetition. For instance, we might repeat layers indexed 3 through 18. Each repeated block has unique LoRA adapters and normalization parameters, and we apply output normalization to repeated blocks after the first repetition. We explore two repetition patterns: (i) **block**: each block is repeated a specified number of times, and (ii) **sequential**: the entire sequence of blocks is repeated in a specified manner. This method allows for efficient extension of smaller models, improving their performance while introducing minimal parameter overhead.

4 Model Compression with FlexiGPT

4.1 Setup

Models - We evaluated our method using LLaMA-2 7B (Touvron et al., 2023), OPT 1.3B and 6.7B (Zhang et al., 2022), and LLaMA-3 8B (AI@Meta, 2024), focusing on these due to their widespread adoption by the community.

Frameworks and resources - Our implementations were done using PyTorch, leveraging FSDP and FP-16 mixed training for efficiency. Experiments were conducted on 4 NVIDIA A100 80GB GPUs, and we utilized the Hugging Face Transformers library for model handling and training. Detailed configurations and additional resources are provided in the Appendix.

Datasets and Benchmarks - We used 1B tokens from the SlimPajama (Soboleva et al., 2023) pretraining dataset for post-prune recovery. For zero-shot performance evaluations, we use the ARC-e, ARC-c (Clark et al., 2018), PIQA (Bisk et al., 2020), WinoGrande (Sakaguchi et al., 2021), and HellaSwag (Zellers et al., 2019) zero-shot benchmarks, utilizing the LM Evaluation Harness (Gao et al., 2021). For perplexity performance evaluations, we use the validation MiniPile (Kaddour, 2023) subset of the Pile dataset (Gao et al., 2020)[3].

Baselines - We compared our method against several baselines, including LLM Surgeon (van der Ouderaa et al., 2024), SliceGPT (Ashkboos et al., 2024), ShortGPT (Men et al., 2024), and ShortGPT + LoRA (an improved version of ShortGPT for a fair comparison with our method). LLM Surgeon and SliceGPT are presented for additional context for experiment results which overlapped with our setting (we use the original results presented in their papers), whereas we implement ShortGPT from scratch for a direct comparison in our setting. LLM-Pruner (Ma et al., 2023) and LaCo (Yang et al., 2024) are not included in our tables as ShortGPT has been found to outperform both methods.

4.2 Results

Main Results - Table 1 summarizes the perplexity (PPL) and zero-shot task performance of various pruning methods on the Llama-2 7B model. Our method, FlexiGPT, shows the lowest PPL of 6.55 at a 30% pruning ratio, outperforming both ShortGPT and ShortGPT + LoRA. In terms of zero-shot task performance, FlexiGPT achieves the highest scores in ARC-c (38.62%), PIQA (74.12%), WinoGrande (66.78%), and HellaSwag (69.02%), with an average performance of 62.68%. This represents a significant improvement over the other methods, demonstrating the effectiveness of our approach. For the 40% pruning ratio, similar trends are observed as FlexiGPT consistently shows superior performance over other methods, achieving the *highest score in every benchmark task*.

Table 2 summarizes the perplexity (PPL) and zero-shot task performance of various pruning methods on the Llama-3 8B model, and Table 3 summarizes the perplexity (PPL) of various pruning methods on the OPT 6.7B and OPT 1.3B models (Zhang et al., 2022). These trends also align with those seen in the Llama-2 7B models, further

[3]We avoid the SlimPajama validation set to avoid giving an unfair advantage to methods trained on the this dataset.